

DataVortex :: Round 2 SQL Queries

Team Name: Fernandoisfasterthanyou

Email: pathakuntlabhaskarreddy@gmail.com

E1: Platform Popularity

```
SELECT
    platform,
    COUNT(*) AS post_count
FROM posts
WHERE platform IS NOT NULL
    AND TRIM(platform) ≠ ''
GROUP BY platform
ORDER BY post_count DESC
LIMIT 1;
```

E2: Most Engaged Posts

```
SELECT
    post_id,
    platform,
    likes,
    shares,
    comments,
    (likes + shares + comments) AS total_engagement
FROM posts
WHERE likes IS NOT NULL
ORDER BY total_engagement DESC
LIMIT 10;
```

E3: Average Engagement by Platform

```
SELECT
    platform,
    ROUND(AVG(COALESCE(likes_cleaned, likes)), 2) AS avg_likes,
    ROUND(AVG(shares), 2) AS avg_shares,
    ROUND(AVG(comments), 2) AS avg_comments,
    ROUND(AVG(COALESCE(likes_cleaned, likes, 0) + shares + comments), 2) AS avg_total_engagement
FROM posts
WHERE platform IS NOT NULL
GROUP BY platform
ORDER BY avg_total_engagement DESC;
```

E4: Highly Shared but Poorly Liked

```
SELECT
    post_id,
    platform,
    likes,
    shares,
    comments
FROM posts
WHERE shares > 1500
    AND likes < 500
    AND likes ≥ 0
ORDER BY shares DESC
LIMIT 15;
```

E5: Users With Large Audiences

```
SELECT
    user_id,
    location,
    language,
    follower_count
FROM users
WHERE follower_count > 40000
ORDER BY follower_count DESC
LIMIT 15;
```

M1: Which Locations Generate the Most Engagement?

```
SELECT
    u.location,
    COUNT(p.post_id) AS post_count,
    SUM(COALESCE(p.likes_cleaned, p.likes, 0) + p.shares + p.comments) AS total_engagement
FROM users u
JOIN posts p ON u.user_id = p.user_id
GROUP BY u.location
ORDER BY total_engagement DESC
LIMIT 15;
```

M2: Do High Follower Users Get More Engagement?

```
SELECT
    CASE
        WHEN u.follower_count ≥ 25000 THEN 'High follower users (≥ 25,000)'
        ELSE 'Low follower users (< 25,000)'
    END AS user_group,
    COUNT(p.post_id) AS total_posts,
    ROUND(AVG(COALESCE(p.likes_cleaned, p.likes, 0) + p.shares + p.comments), 2) AS avg_engagement
FROM users u
JOIN posts p ON u.user_id = p.user_id
GROUP BY
    CASE
        WHEN u.follower_count ≥ 25000 THEN 'High follower users (≥ 25,000)'
        ELSE 'Low follower users (< 25,000)'
    END;
END;
```

M3: Most Active Users

```
SELECT
    u.user_id,
    u.follower_count,
    u.location,
    COUNT(p.post_id) AS post_count,
    SUM(COALESCE(p.likes_cleaned, p.likes, 0) + p.shares + p.comments) AS total_engagement
FROM users u
JOIN posts p ON u.user_id = p.user_id
GROUP BY u.user_id, u.follower_count, u.location
ORDER BY post_count DESC, total_engagement DESC
LIMIT 10;
```

M4: Platform Behaviour by High Follower Users

```
SELECT
    p.platform,
    COUNT(p.post_id) AS post_count,
    ROUND(AVG(COALESCE(p.likes_cleaned, p.likes, 0) + p.shares + p.comments), 2) AS avg_engagement_per_post
FROM users u
JOIN posts p ON u.user_id = p.user_id
WHERE u.follower_count ≥ 30000
    AND p.platform IS NOT NULL
GROUP BY p.platform
ORDER BY avg_engagement_per_post DESC;
```

M5: Detect Suspicious Engagement

```
SELECT
    post_id,
    platform,
    likes,
    shares,
    comments,
    (COALESCE(likes_cleaned, likes, 0) + comments) AS likes_plus_comments,
    (shares - (COALESCE(likes_cleaned, likes, 0) + comments)) AS share_excess
FROM posts
WHERE shares > (COALESCE(likes_cleaned, likes, 0) + comments)
ORDER BY shares DESC
LIMIT 20;
```

H1: Find Users With Abnormally High Engagement

```
WITH UserTotalStats AS (  
    SELECT  
        u.user_id,  
        u.location,  
        u.follower_count,  
        COUNT(p.post_id) AS post_count,  
        ROUND(AVG(COALESCE(p.likes_cleaned, p.likes, 0) + p.shares + p.comments), 2) AS avg_engagement,  
        SUM(COALESCE(p.likes_cleaned, p.likes, 0) + p.shares + p.comments) AS total_engagement  
    FROM users u  
    JOIN posts p ON u.user_id = p.user_id  
    GROUP BY u.user_id, u.location, u.follower_count  
,  
OverallUserAvg AS (  
    SELECT AVG(total_engagement) AS avg_user_total  
    FROM UserTotalStats  
)  
SELECT  
    u.user_id,  
    u.location,  
    u.follower_count,  
    u.post_count,  
    u.avg_engagement,  
    u.total_engagement  
FROM UserTotalStats u  
CROSS JOIN OverallUserAvg o  
WHERE u.total_engagement > 2 * o.avg_user_total  
ORDER BY u.total_engagement DESC  
LIMIT 10;
```

H2: Rank Users Within Their Location

```
WITH UserLocationEngagement AS (  
    SELECT  
        u.location,  
        u.user_id,  
        SUM(COALESCE(p.likes_cleaned, p.likes, 0) + p.shares + p.comments) AS total_engagement  
    FROM users u  
    JOIN posts p ON u.user_id = p.user_id  
    GROUP BY u.location, u.user_id  
)  
  
RankedUsers AS (  
    SELECT  
        location,  
        user_id,  
        total_engagement,  
        DENSE_RANK() OVER (PARTITION BY location ORDER BY total_engagement DESC) AS location_rank  
    FROM UserLocationEngagement  
)  
  
SELECT  
    location,  
    user_id,  
    total_engagement,  
    location_rank  
FROM RankedUsers  
WHERE location_rank ≤ 3  
ORDER BY location, location_rank  
LIMIT 15;
```

H3: Platform Performance Compared With Its Own Average

```
WITH PlatformAvg AS (  
    SELECT  
        platform,  
        AVG(COALESCE(likes, 0) + shares + comments) AS platform_avg_engagement  
    FROM posts  
    WHERE platform IS NOT NULL  
    GROUP BY platform  
)  
,  
PostStats AS (  
    SELECT  
        p.post_id,  
        p.platform,  
        (COALESCE(p.likes, 0) + p.shares + p.comments) AS post_engagement  
    FROM posts p  
    WHERE p.platform IS NOT NULL  
)  
SELECT  
    ps.post_id,  
    ps.platform,  
    ps.post_engagement,  
    ROUND(pa.platform_avg_engagement, 2) AS platform_avg_engagement  
FROM PostStats ps  
JOIN PlatformAvg pa ON ps.platform = pa.platform  
WHERE ps.post_engagement ≥ 2 * pa.platform_avg_engagement  
ORDER BY ps.post_engagement DESC  
LIMIT 10;
```

H4: Follower to Engagement Anomaly

```
WITH UserTotals AS (  
    SELECT  
        u.user_id,  
        u.follower_count,  
        u.location,  
        SUM(COALESCE(p.likes_cleaned, p.likes, 0) + p.shares + p.comments) AS total_engage  
        PERCENT_RANK() OVER (ORDER BY SUM(COALESCE(p.likes_cleaned, p.likes, 0) + p.shares  
    FROM users u  
    JOIN posts p ON u.user_id = p.user_id  
    GROUP BY u.user_id, u.follower_count, u.location  
)  
    )  
SELECT  
    user_id,  
    follower_count,  
    location,  
    total_engagement,  
    ROUND(engagement_percentile * 100, 2) AS engagement_percentile_pct  
FROM UserTotals  
WHERE follower_count < 5000  
    AND engagement_percentile ≥ 0.90  
ORDER BY total_engagement DESC  
LIMIT 10;
```

H5: Identify Data Anomalies (Corrupted Dataset)

```
SELECT post_id, 'Negative Likes' AS anomaly_type
FROM posts_corrupted
WHERE likes < 0

UNION ALL

SELECT post_id, 'Missing Platform' AS anomaly_type
FROM posts_corrupted
WHERE platform IS NULL OR TRIM(platform) = ''

UNION ALL

SELECT post_id, 'Missing Text Content' AS anomaly_type
FROM posts_corrupted
WHERE text_content IS NULL OR TRIM(text_content) = ''

UNION ALL

SELECT post_id, 'HTML Entities or Tags in Text' AS anomaly_type
FROM posts_corrupted
WHERE text_content LIKE '%&%;%'
   OR text_content LIKE '%<div>%'
   OR text_content LIKE '%</div>%'
   OR text_content LIKE '%<br>%'
   OR text_content LIKE '%<br />%'
   OR text_content LIKE '%<br />%'
ORDER BY post_id
LIMIT 15;
```

H6: Find the Most Suspicious High Impact Users

```
WITH OverallMetric AS (  
    SELECT AVG(COALESCE(likes_cleaned, likes, 0) + shares + comments) AS overall_avg_engage  
    FROM posts  
)  
,  
UserMetrics AS (  
    SELECT  
        u.user_id,  
        u.location,  
        u.follower_count,  
        COUNT(p.post_id) AS post_count,  
        AVG(COALESCE(likes_cleaned, likes, 0) + p.shares + p.comments) AS avg_engagement,  
        SUM(COALESCE(likes_cleaned, likes, 0) + p.shares + p.comments) AS total_engagement  
        SUM(CASE WHEN p.shares > COALESCE(likes_cleaned, likes, 0) THEN 1 ELSE 0 END) AS s  
    FROM users u  
    JOIN posts p ON u.user_id = p.user_id  
    GROUP BY u.user_id, u.location, u.follower_count  
)  
SELECT  
    um.user_id,  
    um.location,  
    um.follower_count,  
    um.post_count,  
    ROUND(um.avg_engagement, 2) AS avg_engagement,  
    um.total_engagement,  
    DENSE_RANK() OVER (ORDER BY um.total_engagement DESC) AS rank_by_total_engagement  
FROM UserMetrics um  
CROSS JOIN OverallMetric om  
WHERE um.follower_count < 10000  
    AND um.avg_engagement > om.overall_avg_engagement  
    AND um.suspicious_share_posts ≥ 1  
ORDER BY um.total_engagement DESC  
LIMIT 10;
```